

Riemann_Lab

hprzeta / Riemann_Lab_IA

Analyse des problèmes

compute_zeros_v3 $\rightarrow$ compute_zeros_v4

Phase C — Illinois C/libmpfr · Phase D prévue

Version	v4 — Complément Phase C
Date	24 mai 2026
Auteur	hprzeta
Branche	Riemann_Lab_IA / Riemann_Lab_C
Référence v2 $\rightarrow$ v3	analyse_problemes_v2_v3_phase0.pdf

Résumé exécutif

v3 (Phase 0) a multiplié la vitesse par $\times 5.3$ (1,4 z/s $\rightarrow$ 3,59 z/s) en implémentant la formule Riemann-Siegel, $\theta(t)$ asymptotique, le parallélisme multiprocessing, et la validation Turing-Backlund.

Goulot identifié : l'affinage Illinois (mpmath Python) représente **80–90 %** du temps résiduel. Le GPU GTX 960M n'améliore que la détection $Z(t)$ (10–20 %).

Objectif v4 : porter Illinois en C/libmpfr pour atteindre $\times 5\text{--}10$ supplémentaires $\rightarrow$ **15–20 z/s** et ramener $T=100\,000$ à **2.5–5 h**.

Contents

1	Contexte de la transition v3 → v4	2
1.1	Rappel de l'état v3	2
1.2	Analyse de profil — pourquoi v3 atteint un plateau	2
2	Problèmes identifiés — v3 vers v4	2
2.1	Problème P1 — Goulot Illinois mpmath [HAUTE]	2
2.2	Problème P2 — CUDA fork() incompatible avec multiprocessing [HAUTE]	3
2.3	Problème P3 — Contention BLAS en tests concurrents [MOYENNE]	4
2.4	Problème P4 — nvtop : GPU 0 % dans la liste processus [FAIBLE]	4
2.5	Problème P5 — nvtop crash : Intel HD + mode NVIDIA principal [FAIBLE]	5
2.6	Problème P6 — CLAUDE.md global écrasé par erreur [FAIBLE]	5
2.7	Tableau récapitulatif — tous les problèmes v3→v4	5
3	Architecture cible — compute_zeros_v4	5
3.1	Vue d'ensemble	5
3.2	Branche Git	6
4	Phase C — Illinois en C/libmpfr	7
4.1	Rappel mathématique — méthode Illinois	7
4.2	Code C — illinois_mpfr.c	7
4.3	Makefile	8
4.4	Interface Python via ctypes	9
4.5	Prérequis système	9
5	Formules clés — rappel et corrections	10
5.1	Formules validées — à ne jamais modifier	10
6	Benchmarks et projections de performance	10
6.1	Résultats définitifs v3 — tests séquentiels	10
6.2	Projections v4 — Illinois C	11
7	Étapes de la Phase C — plan d'exécution	11
7.1	Étape suivante — python-flint / Arb	11
8	Infrastructure — état et checklist	11
8.1	Stack logicielle opérationnelle	11
8.2	Checklist post-reboot NVIDIA	11
9	Questions ouvertes	12
9.1	Questions techniques	12
9.2	Questions mathématiques (Niveau 5)	12
10	Références	13

1 Contexte de la transition v3 → v4

1.1 Rappel de l'état v3

La version 3 (`compute_zeros_v3.py`) constitue la version de production actuelle sur la branche `Riemann_Lab_IA`. Elle a calculé avec succès **10 142 zéros non-triviaux** de $\zeta(s)$ sur la droite critique $\operatorname{Re}(s) = \frac{1}{2}$ jusqu'à $T \approx 9998.85$, validés par Turing-Backlund et comparés aux tables LMFDB.

Table 1: Résumé des gains v2 → v3 (rappel)

Métrique	v2	v3	Gain
Durée T=10 000	21 h	~47 min	×26
Vitesse (batch CPU)	0.1 z/s	3.59 z/s	×36
Validation Turing	Absente	N(T) Backlund	—
GPU intégré	Non	GTX 960M + CuPy	—

1.2 Analyse de profil — pourquoi v3 atteint un plateau

Le profilage de v3 révèle la distribution temporelle suivante :

Répartition du temps de calcul — v3

$$t_{\text{total}} = \underbrace{t_{\text{détection}}}_{10-20\%} + \underbrace{t_{\text{Illinois}}}_{80-90\%}$$

Composant	Outil	Accélération GPU
Détection Z(t) — Riemann-Siegel batch	NumPy / CuPy	✓ GPU ×10
Affinage Illinois — méthode sécante modifiée	mpmath Python	CPU pur

Conclusion : accélérer davantage la détection (GPU, batch plus grand) n'apporterait qu'un gain marginal (<20 % du temps total). L'accélération décisive passe par **l'affinage Illinois lui-même**.

2 Problèmes identifiés — v3 vers v4

Cette section catalogue tous les problèmes découverts depuis la finalisation de v3, classés par catégorie.

2.1 Problème P1 — Goulot Illinois mpmath [HAUTE]

P1 — Illinois Python : 80–90 % du temps résiduel

Symptôme : $\text{BATCH_GPU} \approx \text{BATCH_CPU}$ (3.39 vs 3.59 z/s) — la GPU n'apporte aucun gain net.

Cause : L'affinage Illinois utilise `mpmath` à 50 dps, un runtime purement Python (GIL actif, overhead interpréteur, appels C-Python fréquents). L'arithmétique multi-précision `mpmath` est ~5× plus lente que `libmpfr` en C pour le même nombre de bits.

Analyse mathématique de la méthode Illinois

Étant donné a, b tels que $Z(a) \cdot Z(b) < 0$:

$$c = b - Z(b) \cdot \frac{b - a}{Z(b) - Z(a)} \quad (\text{sécante}) \quad (1)$$

$$\text{Si } Z(a) \cdot Z(c) < 0 \Rightarrow b \leftarrow c \quad (2)$$

$$\text{Sinon } \Rightarrow a \leftarrow c, \quad Z(a) \leftarrow Z(a)/2 \quad (\text{correction Illinois}) \quad (3)$$

Convergence : $|b - a| < \varepsilon$ avec $\varepsilon = 10^{-12}$. Nombre d'itérations typique : 8–15 par zéro à 50 dps.

Solution : Phase C — Illinois en C/libmpfr

Solution P1

Porter Illinois en C avec libmpfr (170 bits $\approx$ 51 décimales). Gain attendu : $\times 5\text{--}10$ sur l'affinage $\rightarrow$ vitesse totale 15–20 z/s.

2.2 Problème P2 — CUDA fork() incompatible avec multiprocessing [HAUTE]

P2 — cudaErrorInitializationError dans les workers

Symptôme :

WARNING GPU erreur runtime :

`cudaErrorInitializationError: initialization error`

$\rightarrow$ Bascule automatique sur CPU numpy

Conséquence : les 4 workers parallèles tombent en fallback CPU batch — la GPU est inutilisée dans les runs parallèles.

Cause technique — CUDA et fork() Unix

CUDA ne supporte pas la réinitialisation du contexte GPU après un `fork()` Unix. `multiprocessing.Pool` crée les workers par `fork()` — le contexte CUDA du processus principal est copié mais devient immédiatement invalide dans les workers :

```
Processus principal
  CuPy initialise (CUDA context valide) [OK]
  |_ fork()
  Worker 0  <- copie du contexte CUDA -> invalide [ERREUR]
  Worker 1  <- copie du contexte CUDA -> invalide [ERREUR]
  Worker 2  <- copie du contexte CUDA -> invalide [ERREUR]
  Worker 3  <- copie du contexte CUDA -> invalide [ERREUR]
```

Impact mesuré : comportement identique au benchmark `batch_cpu` (3.5 z/s par worker). Aucune perte de résultat, mais GPU inutilisée.

Solution (Phase C/D)

Solution P2

Initialiser CuPy **après** le `fork()`, à l'intérieur de chaque worker, en passant le contexte GPU par paramètre explicite.

Listing 1: Correction : init CuPy post-fork

```
1  # A implémenter dans parallel_scanner.py
2  def worker_init(gpu_id):
3      """Initialisation apres fork()      CuPy inistialise ici, pas avant.
4      """
5      import cupy as cp
```

```

5     cp.cuda.Device(gpu_id).use()  # contexte GPU valide dans ce worker
6
7     pool = multiprocessing.Pool(
8         processes=4,
9         initializer=worker_init,
10        initargs=(0,)    # GPU 0 = GTX 960M
11    )

```

2.3 Problème P3 — Contention BLAS en tests concurrents [MOYENNE]

P3 — Benchmarks biaisés : 3 modes lancés simultanément

Symptôme (13 mai 2026) : les 3 modes testés concurremment affichent tous des performances dégradées :

Mode	Vitesse mesurée	Vitesse réelle (séquentiel)
CPU scalaire	0.71 z/s	0.67 z/s
BATCH_CPU	0.63 z/s	3.59 z/s
BATCH_GPU	0.60 z/s	3.39 z/s

Facteur de contention mesuré : $\times 2.1$

Cause : BLAS (utilisé par NumPy pour `np.dot()`) crée automatiquement des threads internes (AVX2 / SSE). Avec 3 processus en parallèle, les 4 cœurs sont saturés et les threads se disputent le cache L2/L3.

Solution appliquée :

Solution P3

Tester chaque mode **séparément** après reboot NVIDIA (16 mai 2026). Résultats validés — gains réels confirmés.

2.4 Problème P4 — nvidia-smi : GPU 0 % dans la liste processus [FAIBLE]

P4 — GPU semble inactive alors qu'elle calcule

Symptôme : nvidia-smi affiche GPU%=0 dans la colonne processus mais le graphe historique montre des pics 30–50 %.

Explication — sous-échantillonnage nvidia-smi

nvidia-smi échantillonne toutes les **2 secondes**. Un burst GPU CuPy dure ~ 50 ms. Entre deux échantillons, le burst est terminé et Illinois CPU a repris :

```

Temps      : 0ms      50ms      2000ms (= 1 sample nvidia-smi)
GPU%       : #####.....
Illinois:  ....#####

nvidia-smi sample a 2000ms -> voit Illinois (CPU) -> affiche GPU=0%

```

Règle établie : pour vérifier l'activité GPU, regarder le graphe historique (en haut) — pas la liste des processus (colonne GPU%).

2.5 Problème P5 — nvtop crash : Intel HD + mode NVIDIA principal [FAIBLE]

P5 — nvtop crash après prime-select nvidia

parse_drm_fdinfo_intel: Assertion failed (core dumped)

Cause : la version packagée Ubuntu d'nvtop tente de lire les informations DRM du GPU Intel même quand il est désactivé.

Solution appliquée :

Solution P5

Recompiler nvtop depuis les sources avec support NVIDIA uniquement :

```
cmake .. \
  -DNVIDIA_SUPPORT=ON \
  -DAMDGPU_SUPPORT=OFF \
  -DINTEL_SUPPORT=OFF \
  -DV3D_SUPPORT=OFF \
  -DMSM_SUPPORT=OFF
make -j4 && sudo make install
```

2.6 Problème P6 — CLAUDE.md global écrasé par erreur [FAIBLE]

P6 — Configuration Claude Code corrompue

Symptôme : ~/.claude/CLAUDE.md (26 lignes, instructions générales) avait été écrasé par le CLAUDE.md projet complet (208 lignes), perturbant Claude Code sur tous les projets.

Solution appliquée (23 mai 2026) :

Solution P6 — Hiérarchie CLAUDE.md correcte

Deux fichiers distincts recréés :

- ~/.claude/CLAUDE.md — 26 lignes, instructions légères globales
- ~/projet_zeta/CLAUDE.md — 208 lignes, contexte Riemann complet

Fichier	Rôle	Lignes
~/.claude/CLAUDE.md	Instructions légères — tous projets	26
~/projet_zeta/CLAUDE.md	Contexte Riemann complet	208
src/calculs/optimisation/CLAUDE.md	Phase C — Illinois, ctypes	108
src/calculs/optimisation/c_modules/CLAUDE.md	Règles C — libmpfr, Makefile	97

2.7 Tableau récapitulatif — tous les problèmes v3→v4

3 Architecture cible — compute_zeros_v4

3.1 Vue d'ensemble

Listing 2: Structure cible v4

```
src/calculs/optimisation/
|-- c_modules/
|   |-- illinois_mpfr.c      <- Affinage Illinois (libmpfr, 170 bits)
```

Table 2: Récapitulatif des problèmes v3 → v4

#	Sévérité	Problème	Statut
P1	HAUTE	Illinois Python, overhead interpréteur mpmath = 80- 90 % du temps	En cours
P2	HAUTE	CUDA invalide contexte GPU fork() incompatible multiprocessing	En cours
P3	MOYENNE	Optimisation BLAS / 4 cœurs BLAS tests concurrents	✓ Résolu
P4	FAIBLE	CPU-échantillonnage 2 s vs burst 50 ms 0 % liste processus nvtop	✓ Résolu
P5	FAIBLE	Ne pas passer DRM Intel actif si GPU désactivé crash Intel HD + prime nvidia	✓ Résolu
P6	FAIBLE	Contenu inutile du fichier projet global écrasé	✓ Résolu

```

|-- illinois_mpfr.h
|-- z_function.c          <- Z(t) en C (float64 pour detection)
|-- z_function.h
|-- Makefile              <- compile -> illinois_mpfr.so
'-- test_illinois.py      <- tests validation ctypes
-- theta_rapide.py        <- theta(t) asymptotique (inchange)
-- riemann_siegel.py      <- Z(t) formule RS (inchange)
-- riemann_siegel_batch.py <- Z(t) vectorise NumPy/GPU (inchange)
-- parallel_scanner.py    <- 4 workers + fix fork+CUDA
-- turing_validation.py   <- N(T) Turing-Backlund (inchange)
-- compute_zeros_v3.py    <- Orchestrateur (production)
'-- compute_zeros_v4.py   <- Orchestrateur v4 (Illinois C)

```

3.2 Branche Git

```

git checkout Riemann_Lab_C      # Phase C -- Illinois en C/libmpfr
# Puis merge dans Riemann_Lab_IA apres validation

```

4 Phase C — Illinois en C/libmpfr

4.1 Rappel mathématique — méthode Illinois

La méthode Illinois est une variante de la méthode de la sécante qui évite la stagnation. Elle opère sur la fonction $Z(t)$ de Hardy, réelle sur la droite critique.

Algorithme Illinois — formule complète

Données : a, b tels que $Z(a) \cdot Z(b) < 0$, tolérance $\varepsilon = 10^{-12}$

Itération :

$$c = b - Z(b) \frac{b - a}{Z(b) - Z(a)} \quad (\text{sécante})$$

$$\text{Si } Z(a) \cdot Z(c) < 0 \Rightarrow b \leftarrow c, \quad Z_b \leftarrow Z(c) \quad (4)$$

$$\text{Sinon } \Rightarrow a \leftarrow c, \quad Z_a \leftarrow Z(c), \quad Z_a \leftarrow Z_a/2 \quad (\text{correction Illinois})$$

Arrêt : $|b - a| < \varepsilon$.

Précision requise : 50 dps = 170 bits (libmpfr : PREC = 170).

4.2 Code C — illinois_mpfr.c

Listing 3: Squelette illinois_mpfr.c

```

1  #include <mpfr.h>
2  #include <stdio.h>
3  #include <math.h>
4
5  #define PREC      170    /* 170 bits ~ 51 decimales */
6  #define MAX_ITER 100
7
8  /* theta(t) asymptotique -- Stirling */
9  static void theta_mpfr(mpfr_t result, mpfr_t t) {
10     mpfr_t tmp, pi, t_sur_2pi, lnt;
11     mpfr_inits2(PREC, tmp, pi, t_sur_2pi, lnt, NULL);
12     mpfr_const_pi(pi, MPFR_RNDN);
13     /* theta = (t/2)*ln(t/2pi) - t/2 - pi/8 + 1/(48t) */
14     mpfr_mul_ui(t_sur_2pi, pi, 2, MPFR_RNDN);
15     mpfr_div(t_sur_2pi, t, t_sur_2pi, MPFR_RNDN);
16     mpfr_log(lnt, t_sur_2pi, MPFR_RNDN);
17     mpfr_mul(result, t, lnt, MPFR_RNDN);
18     mpfr_div_ui(result, result, 2, MPFR_RNDN);
19     mpfr_div_ui(tmp, t, 2, MPFR_RNDN);
20     mpfr_sub(result, result, tmp, MPFR_RNDN);
21     mpfr_div_ui(tmp, pi, 8, MPFR_RNDN);
22     mpfr_sub(result, result, tmp, MPFR_RNDN);
23     mpfr_clears(tmp, pi, t_sur_2pi, lnt, NULL);
24 }
25
26 /* Z(t) via formule Riemann-Siegel, PREC bits */
27 static void Z_mpfr(mpfr_t result, mpfr_t t) {
28     mpfr_t theta, sum, term, n_val, cos_arg, sqrtn;
29     mpfr_inits2(PREC, theta, sum, term, n_val, cos_arg, sqrtn, NULL);
30     theta_mpfr(theta, t);
31     mpfr_t pi2, t_over_2pi;
32     mpfr_inits2(PREC, pi2, t_over_2pi, NULL);
33     mpfr_const_pi(pi2, MPFR_RNDN);
34     mpfr_mul_ui(pi2, pi2, 2, MPFR_RNDN);

```



```

35 mpfr_div(t_over_2pi, t, pi2, MPFR_RNDN);
36 mpfr_sqrt(t_over_2pi, t_over_2pi, MPFR_RNDN);
37 long N = mpfr_get_si(t_over_2pi, MPFR_RNDZ);
38 mpfr_set_ui(sum, 0, MPFR_RNDN);
39 for (long n = 1; n <= N; n++) {
40     mpfr_set_si(n_val, n, MPFR_RNDN);
41     mpfr_log(cos_arg, n_val, MPFR_RNDN); /* ln(n) */
42     mpfr_mul(cos_arg, t, cos_arg, MPFR_RNDN);
43     mpfr_sub(cos_arg, theta, cos_arg, MPFR_RNDN);
44     mpfr_cos(term, cos_arg, MPFR_RNDN);
45     mpfr_sqrt(sqtrn, n_val, MPFR_RNDN);
46     mpfr_div(term, term, sqtrn, MPFR_RNDN);
47     mpfr_add(sum, sum, term, MPFR_RNDN);
48 }
49 mpfr_mul_ui(result, sum, 2, MPFR_RNDN);
50 mpfr_clears(theta, sum, term, n_val, cos_arg, sqtrn, pi2,
51             t_over_2pi, NULL);
52
53 /* illinois_mpfr : retourne Im(zero) dans [a_d, b_d] */
54 double illinois_mpfr(double a_d, double b_d, double tol) {
55     mpfr_t a, b, c, Za, Zb, Zc, fa, diff, num, den;
56     mpfr_inits2(PREC, a, b, c, Za, Zb, Zc, fa, diff, num, den, NULL);
57     mpfr_set_d(a, a_d, MPFR_RNDN);
58     mpfr_set_d(b, b_d, MPFR_RNDN);
59     Z_mpfr(Za, a);
60     Z_mpfr(Zb, b);
61     for (int i = 0; i < MAX_ITER; i++) {
62         /* c = b - Zb*(b-a)/(Zb-Za) */
63         mpfr_sub(diff, b, a, MPFR_RNDN);
64         mpfr_mul(num, Zb, diff, MPFR_RNDN);
65         mpfr_sub(den, Zb, Za, MPFR_RNDN);
66         mpfr_div(num, num, den, MPFR_RNDN);
67         mpfr_sub(c, b, num, MPFR_RNDN);
68         Z_mpfr(Zc, c);
69         mpfr_sub(diff, b, a, MPFR_RNDN);
70         mpfr_abs(diff, diff, MPFR_RNDN);
71         if (mpfr_get_d(diff, MPFR_RNDN) < tol) break;
72         mpfr_mul(fa, Za, Zc, MPFR_RNDN);
73         if (mpfr_sgn(fa) < 0) {
74             mpfr_set(b, c, MPFR_RNDN); mpfr_set(Zb, Zc, MPFR_RNDN);
75         } else {
76             mpfr_set(a, c, MPFR_RNDN); mpfr_set(Za, Zc, MPFR_RNDN);
77             mpfr_mul_d(Za, Za, 0.5, MPFR_RNDN); /* correction Illinois */
78         }
79     }
80     double result = mpfr_get_d(c, MPFR_RNDN);
81     mpfr_clears(a, b, c, Za, Zb, Zc, fa, diff, num, den, NULL);
82     return result;
83 }

```

4.3 Makefile

Listing 4: c_modules/Makefile

```

CC = gcc
CFLAGS = -O3 -march=native -fPIC -Wall
LIBS = -lmpfr -lgmp -lm

```

```
all: illinois_mpfr.so

illinois_mpfr.so: illinois_mpfr.c z_function.c
    $(CC) $(CFLAGS) -shared -o $@ $^ $(LIBS)

test: test_illinois.py illinois_mpfr.so
    python3 test_illinois.py

clean:
    rm -f *.so *.o
```

4.4 Interface Python via ctypes

Listing 5: test_illinois.py — interface ctypes

```
1 import ctypes, os
2
3 # Charger la bibliothèque compilée
4 lib = ctypes.CDLL(
5     os.path.join(os.path.dirname(__file__), 'illinois_mpfr.so')
6 )
7 lib.illinois_mpfr.restype = ctypes.c_double
8 lib.illinois_mpfr.argtypes = [
9     ctypes.c_double, # a : borne gauche
10    ctypes.c_double, # b : borne droite
11    ctypes.c_double # tol : tolerance (ex. 1e-12)
12 ]
13
14 def illinois_c(a: float, b: float, tol: float = 1e-12) -> float:
15     """Affinage Illinois en C/libmpfr -- x5 vs mpmath."""
16     return lib.illinois_mpfr(a, b, tol)
17
18 # --- Validation : 3 premiers zeros ---
19 REF = [14.134725141734693, 21.022039638771555, 25.010857580145688]
20 tests = [(14.0, 14.3), (21.0, 21.1), (25.0, 25.1)]
21
22 for i, (a, b) in enumerate(tests):
23     z = illinois_c(a, b)
24     ecart = abs(z - REF[i])
25     ok = "OK" if ecart < 1e-11 else "ERREUR"
26     print(f"Zero {i+1}: {z:.15f} ecart={ecart:.2e} [{ok}]")
```

4.5 Prérequis système

```
# Ubuntu 22.04 / 24.04
sudo apt install libmpfr-dev libgmp-dev gcc build-essential

# Vérifier
mpfr-config --version # >= 4.0
gcc --version
```

5 Formules clés — rappel et corrections

5.1 Formules validées — à ne jamais modifier

Formule $N(T)$ — CORRECTE

$$N(T) = \frac{T}{2\pi} \ln \frac{T}{2\pi e}$$

Le **facteur** e dans $2\pi e$ est **obligatoire**. Sans lui :

T	$N(T)$ correct	Sans e	Erreur
10 000	10 142	~7 300	−28 %
100 000	138 067	~49 346	−64 %

STEP sécurisé — évite les zéros manquants

$$\text{STEP} = \min\left(\frac{2\pi}{5 \ln(T_{\max}/2\pi)}, 0.10\right)$$

Formule de Riemann-Siegel

$$Z(t) = 2 \sum_{n=1}^N \frac{\cos(\theta(t) - t \ln n)}{\sqrt{n}} + R(t), \quad N = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor$$

$\theta(t)$ asymptotique — développement de Stirling

$$\theta(t) = \frac{t}{2} \ln \frac{t}{2\pi} - \frac{t}{2} - \frac{\pi}{8} + \frac{1}{48t} + \frac{7}{5760 t^3} + O(t^{-5})$$

Validation Turing — Riemann–von Mangoldt

$$N(T) = \frac{\theta(T)}{\pi} + 1 + S(T), \quad S(T) = \frac{1}{\pi} \arg \zeta\left(\frac{1}{2} + iT\right)$$

6 Benchmarks et projections de performance

6.1 Résultats définitifs v3 — tests séquentiels

Table 3: Benchmarks v3 — tests séquentiels après reboot NVIDIA (16 mai 2026)

Mode	Zéros/15 min	Vitesse	t atteint	Gain
CPU scalaire (référence)	604	0.67 z/s	944	—
BATCH_CPU	3231	3.59 z/s	4164	×5.3
BATCH_GPU	3051	3.39 z/s	4596	×5.1

Observation clé : BATCH_GPU $\approx$ BATCH_CPU — la GTX 960M n’apporte pas de gain net car Illinois (80–90 % du temps) reste sur CPU.

Table 4: Projections v4 après Phase C (Illinois C/libmpfr)

Composant	v3 (mpmath)	v4 (C/libmpfr)	Gain
Illinois affinage	80–90 % du temps	~15–20 %	×5–10
Détection $Z(t)$ BATCH	3.59 z/s	inchangé	—
Total estimé	3.59 z/s	15–20 z/s	×4–6

Table 5: Temps de calcul projetés par T_MAX

T_MAX	N(T)	v2 (21 h/10k)	v3 batch	v4 (×5–10)
10 000	10 142	21 h	~47 min	~8–10 min
100 000	138 067	~30 j	~11 h	1.5–2.5 h
500 000	750 000	—	~2.5 j	5–12 h

6.2 Projections v4 — Illinois C

7 Étapes de la Phase C — plan d’exécution

1. **Implémenter** `illinois_mpr.c` avec $\theta(t)$ Stirling et $Z(t)$ Riemann-Siegel en libmpfr (170 bits).
2. **Valider** sur les 10 premiers zéros — écart $< 10^{-11}$ vs LMFDB.
3. **Benchmark** Illinois C vs Illinois mpmath sur 100 zéros — mesurer le $\times$ réel.
4. **Intégrer** dans `compute_zeros_v4.py` via ctypes (fallback mpmath si `.so` absent).
5. **Fix fork+CUDA** dans `parallel_scanner.py` (init CuPy post-fork).
6. **Valider** T=1000 complet avec Turing-Backlund.
7. **Rapport** v3→v4 : `.md` + PDF `pdflatex`.
8. **Merge** `Riemann_Lab_C` → `Riemann_Lab_IA` après validation.

7.1 Étape suivante — python-flint / Arb

Après libmpfr : utiliser `python-flint` (bindings Python de FLINT/Arb) pour une arithmétique certifiée (bornes d’erreur garanties) — base de la méthode Odlyzko-Schönhage partielle.

```
pip install python-flint # Arb inclus
```

8 Infrastructure — état et checklist

8.1 Stack logicielle opérationnelle

8.2 Checklist post-reboot NVIDIA

```
# 1. Confirmer GPU active
nvidia-smi

# 2. Benchmark BATCH_CPU seul (15 min)
python benchmark_15min.py --mode batch_cpu --tmax 10000 --duree 15

# 3. Benchmark BATCH_GPU seul (15 min)
python benchmark_15min.py --mode batch_gpu --tmax 10000 --duree 15

# 4. Comparer vs CPU scalaire (~0.67 z/s)
```

Table 6: État de la stack — 24 mai 2026

Composant	Version / Note	Statut
Python	3.12, venv <code>zeta_env</code>	✓
mpmath	50 dps — Illinois (Python)	✓
NumPy	BLAS AVX2	✓
CuPy	<code>cupy-cuda12x</code> (CUDA 12.2)	✓
GTX 960M	640 cœurs CUDA, 4 GB VRAM	✓
nvtop	Recompilé — NVIDIA seul	✓
Ollama	3 modèles sur <code>/mnt/data</code>	✓
CLAUDE.md	4 niveaux hiérarchiques	✓
libmpfr-dev	À installer	En cours
illinois_mpfr.so	À compiler (Phase C)	En cours

```
# 5. Lancer compute_zeros_v3 T=10000 avec validation Turing complete

# 6. Installer pr requis Phase C
sudo apt install libmpfr-dev libgmp-dev gcc build-essential

# 7. Commencer Phase C
git checkout Riemann_Lab_C
cd src/calculs/optimisation/c_modules/
# -> implementer illinois_mpfr.c
```

9 Questions ouvertes

9.1 Questions techniques

1. **Gain réel Illinois C** : le $\times 5\text{--}10$ théorique sera-t-il confirmé par le benchmark ? (overhead ctypes, cache mpfr, etc.)
2. **Fork + CUDA** : la correction `initializer` résoudra-t-elle pleinement le `cudaErrorInitializationError` sans réduire la vitesse du worker principal ?
3. **T = 100 000** : le STEP adaptatif est-il suffisant à grand T , ou faut-il un maillage variable basé sur $\text{gap}(T) \sim 2\pi/\ln(T/2\pi)$?
4. **python-flint vs libmpfr pur** : l'arithmétique intervalles d'Arb permet-elle de garantir l'absence de zéros manquants sans Turing ?

9.2 Questions mathématiques (Niveau 5)

1. La conjecture de Montgomery sur la corrélation des paires de zéros : confirmée numériquement jusqu'à quel T avec les données actuelles ?
2. Les fluctuations de $S(T) = \frac{1}{\pi} \arg \zeta(\frac{1}{2} + iT)$ au-delà de $T = 10\,000$ — distribution GUE confirmée ?
3. Implémentation de la formule explicite de Riemann : $\psi_0(x) = x - \sum_{\rho} \frac{x^{\rho}}{\rho} - \ln(2\pi)$ avec les 10 142 zéros calculés — comparaison avec $\pi(x)$?

10 Références

- Titchmarsh, E.C. (1986). *The Theory of the Riemann Zeta-Function*, §4.12. <https://archive.org/details/theoryofriemann00titchmarsh/page/n3-m0>
- Turing, A.M. (1953). Some calculations of the Riemann zeta-function. *Proc. London Math. Soc.* **3**, 99–117. <https://doi.org/10.1112/plms/s3-3.1.99>
- Odlyzko, A.M. & Schönhage, A. (1988). Fast algorithms for multiple evaluations of the Riemann zeta function. *Trans. Amer. Math. Soc.* **309**, 797–809. <https://doi.org/10.1090/S0002-9947-1988-0936813-0>
- LMFDB — Zeros of $\zeta(s)$: <https://lmfdb.org/zeros/zeta/>
- libmpfr documentation : <https://www.mpfr.org/mpfr-current/mpfr.html>
- Rapport précédent v2→v3 : [pdf/optimisation/analyse_problemes_v2_v3_phase0.pdf](#)